

TRAIN SHORT, INFER LONG: SPEECH-LLM ENABLES ZERO-SHOT STREAMABLE JOINT ASR AND DIARIZATION ON LONG AUDIO

Mohan Shi^{1*}, Xiong Xiao², Ruchao Fan², Shaoshi Ling², Jinyu Li²

¹University of California, Los Angeles, USA

²Microsoft CoreAI, Redmond, USA

ABSTRACT

Joint automatic speech recognition (ASR) and speaker diarization aim to answer the question “who spoke what” in multi-speaker scenarios. In this paper, we present an end-to-end speech large language model (Speech-LLM) for **Joint streamable Diarization and aSr** (JEDIS-LLM). The model is trained only on short audio under 20s but is capable of streamable inference on long-form audio without additional training. This is achieved by introducing a Speaker Prompt Cache (SPC) with an on-the-fly update mechanism during chunk-wise streaming inference, inspired by the autoregressive nature of LLMs. The SPC also allows the seamless use of pre-enrolled speaker profiles which is common in many scenarios like meeting transcription. To further enhance diarization capability, we incorporate word-level speaker supervision into the speech encoder during training.

Experimental results demonstrate that our system outperforms strong baselines, including Sortformer and Meta-Cat in the local setting on audio up to 20s, and DiarizationLM on long-form audio, despite being fully end-to-end and streamable while DiarizationLM follows a cascaded offline pipeline. To the best of our knowledge, this is the first work enabling zero-shot streamable joint ASR and diarization on long audio using a Speech-LLM trained only on short audio, achieving state-of-the-art performance.

Index Terms— Speech-LLM, ASR, Speaker Diarization, Speaker Prompt Cache, Long-form Audio, Streaming

1. INTRODUCTION

With advances in deep learning and automatic speech recognition (ASR) [1–3], multi-speaker scenarios such as meetings and conversations have received increasing attention. Speaker diarization [4–6] aims to identify “who spoke when” in a recording. Combined with ASR, it forms the joint ASR and diarization task [7,8], also known as speaker-attributed ASR [8–11], which addresses “who spoke what” and is crucial for understanding multi-speaker conversations.

Previous works combined independent ASR and diarization outputs to obtain speaker-attributed transcriptions [12–14], but such cascaded systems suffer from error propagation. Recent approaches, such as Sortformer [15] and Meta-Cat [16], integrate speaker IDs directly into multi-talker transcriptions, enabling end-to-end training and achieving reasonable performance on short audio (up to 20 s). However, these methods still rely on two separate pre-trained encoders for ASR and diarization, whose representations are fused during decoding. More importantly, they are not suitable for long-form audio that requires global diarization. Streaming Sortformer [17] introduces a cache mechanism for global diarization, but it does not incorporate the ASR task.

The long-context reasoning and cross-utterance awareness abilities of large language models (LLMs) [18–20] make them well-suited for multi-talker conversations. Prior works [21, 22] showed that Speech-LLMs perform well on multi-talker ASR, but without incorporating speaker diarization. The recent DiarizationLM [23] is a post-processing LLM that refines outputs from independent ASR and diarization systems to produce better speaker-attributed transcriptions. However, it is not end-to-end and suffers from error propagation. Concurrently, SpeakerLM [24] trains a Speech-LLM jointly for ASR and diarization in an end-to-end manner, yet it is limited to short audio in local settings and was not compared against recent strong baselines [15, 16, 23].

In this paper, we propose an end-to-end Speech-LLM for **Joint streamable Diarization and aSr** (JEDIS-LLM). Although trained only on short audio (≤ 20 s), the model supports chunk-wise streaming inference on arbitrary long-form recordings without additional training. A central challenge of long audio diarization is that splitting audio into short chunks often causes speaker permutation inconsistency [25]. Traditional methods address this with post-hoc global clustering [26]. Inspired by prior works [17, 25] and the autoregressive property of LLMs, we introduce the **Speaker Prompt Cache (SPC)** with an on-the-fly update mechanism for chunk-wise streaming inference that preserves speaker consistency across chunks without explicit permutation resolution or retraining. SPC stores a representative utterance for each speaker and combines it with the current chunk during inference. It also naturally supports pre-enrolled **Speaker Profiles**, which are commonly used in scenarios such as meeting transcription. During training, we enhance the diarization capability of our proposed model by introducing **Word-level Speaker Supervision** to the speech encoder, in contrast to prevailing Speech-LLM architectures [21, 27] for ASR that consist only of a speech encoder, a projector, and an LLM.

Experimental results show that our approach outperforms strong baselines, including Sortformer [15] and Meta-Cat [16] in the local setting (audio up to 20s), and DiarizationLM [23] on long-form audio, while remaining fully end-to-end and streamable, unlike DiarizationLM’s cascaded offline pipeline. To the best of our knowledge, this is the first work to achieve zero-shot, streamable joint ASR and diarization on arbitrary long-form audio using a Speech-LLM trained only on short clips, achieving state-of-the-art results.

2. METHOD

2.1. Training on Short Audio

As shown in Figure 1 (a), our proposed JEDIS-LLM is built on the prevailing Speech-LLM architecture [21, 27] with speaker-attributed transcription as the LLM objective. The key difference is the addition of a **Spk-Decoder** for speaker supervision to enhance diarization capability. We describe these components separately below.

*Work done during an internship at Microsoft.

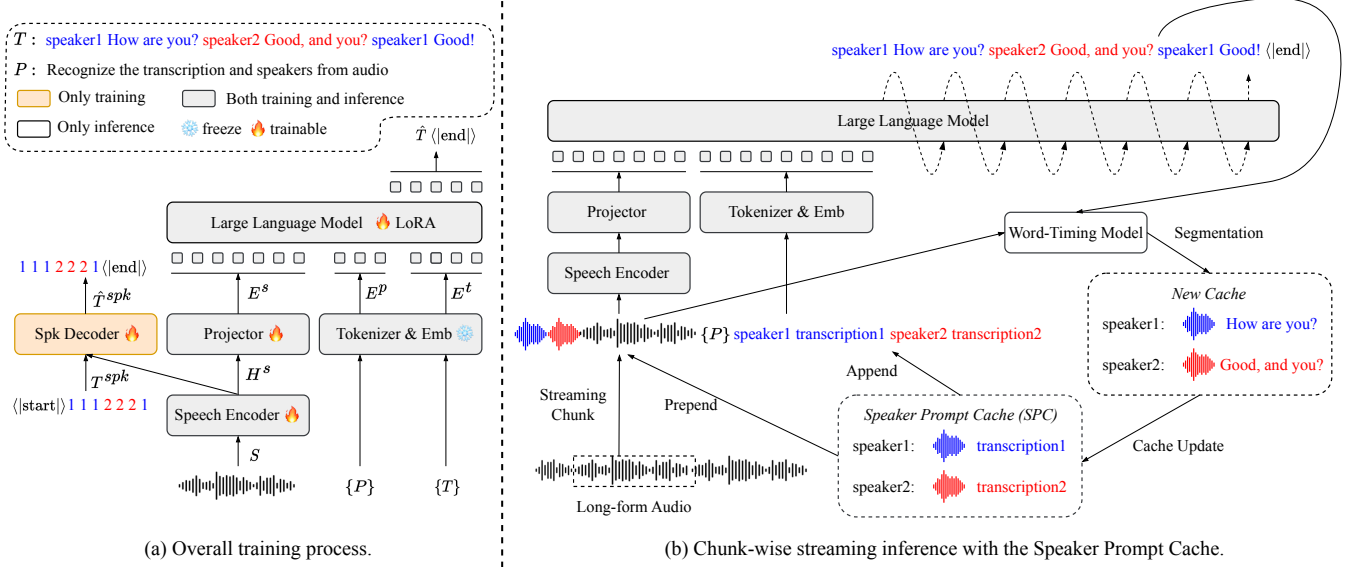


Fig. 1: Overall training pipeline (a) and chunk-wise streaming inference using the Speaker Prompt Cache (b). The Word-Timing Model provides timestamp alignment of each word for segmentation. SPC is not required for offline inference on short audio.

2.1.1. Speech-LLM for joint ASR and Diarization

We construct speaker-attributed transcriptions by integrating multi-talker transcriptions with speaker IDs, which serve as the LLM training objective. For multi-speaker utterances, words are arranged in temporal order, with a speaker ID inserted whenever the speaker changes, thus forming the *segment-level* objective. In contrast, the *word-level* objective inserts a speaker ID before every word [15, 16], which under-utilizes the contextual modeling ability of LLMs and slows inference due to longer sequence. Therefore, we adopt the *segment-level* objective in this work. Given a speech signal S , the forward process is defined as:

$$H^s = \text{Speech-Encoder}(S), \quad E^s = \text{Projector}(H^s), \quad (1)$$

$$E^t = \text{Emb}(\text{Tokenizer}(T)), \quad E^p = \text{Emb}(\text{Tokenizer}(P)), \quad (2)$$

$$\hat{T} = \text{LLM}(\text{Concat}(E^s, E^p, E^t)), \quad (3)$$

where T denotes the speaker-attributed transcription and P is the text prompt. The **Tokenizer** and **Emb** correspond to the LLM’s tokenizer and text embedding layers. An additional LoRA [28] is introduced to adapt the LLM outputs to the format required for joint ASR and diarization. Finally, the Cross-Entropy (CE) loss is computed between the predicted sequence $\hat{T}$ and the ground truth T :

$$\mathcal{L}_{\text{LLM}} = \text{CE}(\hat{T}, T). \quad (4)$$

2.1.2. Word-level Speaker Supervision for Speech Encoder

To enhance the speaker diarization capability of the speech encoder, we introduce additional speaker supervision during training to encourage the encoder to learn speaker-discriminative features. While prior works typically adopt **frame-level** multi-class binary classification loss for speaker diarization [5, 15], we observe that this approach can harm ASR performance in the joint ASR and diarization task, as frame-level labels lack semantic information. Moreover, such labels are obtained from forced alignment and often contain annotation errors. To address this, we propose a new scheme, termed **Word-level Speaker Supervision** (distinct from the “word-level” objective in Section 2.1.1). As illustrated in Figure 1 (a), each word in the transcription is replaced with its corresponding speaker ID, forming a

word-level speaker ID sequence. This sequence is predicted by a transformer-based **Spk-Decoder** applied to the encoder output. Finally, the CE loss is computed between the predicted and reference sequences:

$$\hat{T}^{spk} = \text{Spk-Decoder}(T^{spk}, H^s), \quad \mathcal{L}_{\text{Spk}} = \text{CE}(\hat{T}^{spk}, T^{spk}), \quad (5)$$

where T^{spk} and $\hat{T}^{spk}$ denote the reference and predicted speaker ID sequences, respectively.

Since speaker supervision is an auxiliary task aimed at enhancing the encoder’s diarization capability, the **Spk-Decoder** is used only during training and discarded at inference. The overall objective is a weighted sum of the LLM token prediction loss and the speaker supervision loss:

$$\mathcal{L} = \mu \cdot \mathcal{L}_{\text{LLM}} + (1 - \mu) \cdot \mathcal{L}_{\text{Spk}}, \quad (6)$$

where μ is a hyperparameter that balances the two losses.

2.2. Inference on Long Audio

2.2.1. SPC for Streaming Inference on Long-Form Audio

For long-form global diarization, inference on short chunks may cause speaker permutation inconsistencies [25], where the same speaker is assigned different labels across chunks. Traditional methods resolve this with global clustering as a post-processing step [26].

We propose an alternative: the **Speaker Prompt Cache (SPC)** for chunk-wise streaming inference. As illustrated in Figure 1 (b), during inference, SPC preserves speaker consistency without explicit permutation resolution by storing one utterance (audio clip and its transcription) for each previously observed speaker.

During chunk-wise inference, leveraging the autoregressive nature of the LLM, we prepend cached audio clips to the current chunk and append cached speaker-attributed transcriptions to the prompt as initial context, ordered by speaker index. This operation provides an exact speaker permutation that serves as the condition for LLM inference, allowing the LLM to follow the same permutation and generate speaker-attributed transcriptions for the current chunk while maintaining speaker consistency.

Additionally, we design a cache update algorithm to maintain the SPC during inference, detailed in Algorithm 1.

Table 1: Performance comparison of different methods in the local setting (audio up to 20s), reported in WDER (%) and cpWER (%). All inference is non-streaming. Phi-4-Multimodal baseline attributes all hypotheses to “speaker1” as it does not support speaker diarization.

System	LLM Objective	Speaker Supervision for Speech Encoder	AMI Test		CH109 Full		Internal Test Set	
			WDER	cpWER	WDER	cpWER	WDER	cpWER
Sortformer [15]	-	-	-	26.71	-	21.45	-	-
Meta-Cat [16]	-	-	-	26.02	-	26.17	-	-
Phi-4-Multimodal ¹ [29]	-	-	14.52	28.09	17.25	33.09	14.68	31.10
JEDIS-LLM (Ablation)	Segment-level	None	10.87	26.00	3.67	19.90	7.27	23.92
	Segment-level	Frame-level	8.01	35.67	2.49	25.08	2.44	24.34
	Word-level	Word-level	6.34	24.08	2.40	24.55	2.65	18.77
JEDIS-LLM (Final Model)	Segment-level	Word-level	6.97	23.13	2.06	19.46	2.49	18.14

2.2.2. Seamless Integration with Speaker Profiles

Building on the **SPC**, we enable seamless integration of **Speaker Profiles** during inference by replacing the SPC with fixed, manually segmented audio clips and their transcriptions, which serve as speaker profiles. This design provides two main benefits:

- The exact speaker names can be retrieved through the speaker-profile mapping. For example, a profile map may be {speaker1: Mike, speaker2: Susan}.
- Using fixed high-quality audio clips and transcriptions instead of on-the-fly SPC eliminates the need for cache updates, yielding more stable performance.

3. EXPERIMENTAL SETTINGS

3.1. Training Datasets

We trained the proposed JEDIS-LLM on five data sources. Public datasets include the AMI Corpus [30] (train and dev sets, IHM-Mix channel, up to 4 speakers/session, ~90 h), the ICSI Corpus [31] (IHM-Mix channel, all subsets, up to 11 speakers/session, ~71 h), and the Fisher Corpus [32] (2 speakers/session, ~1929 h). We also incorporated internally collected data (up to 7 speakers/session, ~6734 h) and simulated conversations from VoxCeleb1 [33] and VoxCeleb2 [34] (~964 h). For the simulations, we removed non-English utterances using language identification [35, 36], and mixed 5 speakers per conversation, each contributing 3–5 sentences with mild overlap ($\leq 1\%$) and room impulse responses up to 0.2 s. In total, the training data amounts to about 10k hours.

3.2. Training Setting

Our model builds on Phi-4-Multimodal¹ [29], using its speech branch as initialization of speech encoder, projector and LLM. The additional LoRA is configured with $\alpha = 32$ and $rank = 16$. The Spk-Decoder has 3 transformer layers with 1024-dimensional hidden states, 16 attention heads, and 1024-dimensional feed-forward layers. The loss weight μ is set to 0.5. Long-form audio is randomly segmented into 15~20s clips for training. The model is trained on 16 NVIDIA A100 80GB GPUs with 256s per GPU batch size, using AdamW optimizer (peak learning rate 0.0001) with linear warmup-decay scheduling (1000 warmup steps, 40,000 steps total).

3.3. Evaluation Setting

We evaluate our JEDIS-LLM under both **local** (short audio) and **global** (long audio) settings. In the local setting, following prior works [15, 16], we evaluate on two datasets: the AMI-IHM-Mix test set (*AMI Test*; up to 4 speakers per session) and the 2-speaker subset of 109 sessions from the Callhome American English Speech corpus (*CH109 Full*) [37]. In both datasets, long-form audio is segmented

Algorithm 1: Streaming Inference with Speaker Prompt Cache

Input: Long audio A , well-trained model M , *prompt*
Output: Speaker-attributed transcriptions R
Initialize empty speaker prompt cache C , result list R
Define profile audio length threshold l , text length threshold n ;
Define dvector similarity threshold θ ;
for each *chunk* a **in** A **do**
 if $C = \emptyset$ **then**
 $a_{\text{infer}} \leftarrow a$, $p_{\text{infer}} \leftarrow \text{prompt}$;
 else
 $a_{\text{infer}} \leftarrow \{C[s].\text{audio}, \forall s \in C\} + a$;
 $p_{\text{infer}} \leftarrow \text{prompt} + \{C[s].\text{text}, \forall s \in C\}$;
 $r \leftarrow M(a_{\text{infer}}, p_{\text{infer}})$, Append r to R ;
 for each *speaker* s **in** r **do**
 $Ali \leftarrow \text{WordTimingModel}(a, s.\text{text})$;
 $(A_s, T_s) \leftarrow \text{Segmentation}(Ali, \text{len} \leq l, \text{exclude overlap})$;
 $(\hat{a}_s, \hat{t}_s) \leftarrow \text{FindLongest}(A_s, T_s)$;
 if $s \notin C$ **then**
 $C[s].\text{audio} \leftarrow \hat{a}_s$, $C[s].\text{text} \leftarrow \hat{t}_s$;
 else if $\text{len}(C[s].\text{text}) < n$ **or** $\neg \text{HasPunctuation}(C[s].\text{text})$ **then**
 if $\text{len}(\hat{a}_s) > \text{len}(C[s].\text{audio})$ **then**
 $\sigma \leftarrow \text{CosSim}(\text{dvector}(\hat{a}_s), \text{dvector}(C[s].\text{audio}))$;
 if $\sigma > \theta$ **then**
 $C[s].\text{audio} \leftarrow \hat{a}_s$, $C[s].\text{text} \leftarrow \hat{t}_s$;
 return R ;

into 10~20s clips. We additionally evaluate on an internal test set (with up to 5 speakers per session).

For the global setting, following prior work [23], we use test subset of CH109 and Fisher, referred to as *CH109 Test* and *Fisher Test*. In chunk-wise streaming inference, we consider **Oracle Chunks**, derived from ground-truth sentence boundaries, and **VAD Chunks**, obtained via voice activity detection². Chunks are up to 10 seconds long, and regions without transcriptions at the beginning and end of the long audio are excluded. The dvector extractor in Algorithm 1 is a well-trained Res2Net [38] trained for speaker verification, and the Word-Timing Model is an internal forced alignment model. The profile audio length threshold l and text length threshold n are set to 5 seconds and 8, respectively. The dvector similarity threshold θ is set to 0.7. For speaker profile integration, we evaluate on *CH109 Test* by extracting audio clips shorter than 5 seconds from non-transcribed portions and manually annotating them as speaker profiles.

We report **Word Diarization Error Rate (WDER)** [7] and **concatenated minimum-permutation Word Error Rate (cp-WER)** [39] as primary metrics. Pure WER is ambiguous in multi-speaker scenarios and is thus excluded. For speaker profiles, we also report **Speaker-Attributed WER (SA-WER)** [8], which directly

¹<https://huggingface.co/microsoft/Phi-4-multimodal-instruct>

²<https://github.com/snakers4/silero-vad>